

Coding with AI: From Writing to Guiding Code

How to use AI tools without losing control of your code





Defining the AI-Assisted Development Workflow

Intent Over Syntax

The primary task of modern coding is now
describing intent rather than manual typing.

Guiding the AI to build the right thing is the new core competency.



3 Tiers of AI Integration in Programming



Chat-Based Tools

- **External interfaces** like ChatGPT
- Manual **copy-pasting** of code
- Limited to **provided snippets**
- Best for **isolated logic**



IDE Copilots

- **Inline suggestions** as you type
- Access to **local file context**
- Significant **speed improvements**
- Integrated into **VS Code/Cursor**



Agentic Systems

- **Autonomous** project actions
- Modifies **multiple files** at once
- High **architectural awareness**
- Requires **strict oversight**

CLI vs IDE



Chat-Based Tools



ChatGPT ▾

↑ Share ...

How do I split fastq.gz files (of DNA short reads) into subfiles, each with 10k reads?

A FASTQ record is 4 lines, so 10k reads = 40k lines per chunk. You must preserve gzip handling and avoid breaking records.

Simple Unix approach (streaming, no full decompression to disk)

`↗` Bash



```
zcat input.fastq.gz | split -l 40000 -d -a 4 - chunk_
```

- `-l 40000` → 40k lines = 10k reads
- `-d` → numeric suffixes
- `-a 4` → 4-digit suffixes (chunk_0000, ...)



Then recompress:

+ Ask anything



IDE Copilots

```
65 def append_full_path(  
66     parent_dir: str,  
67     post_clean_dir: str,  
68     project_paths: List[str],  
69     run_name: str,  
70 ):  
71     """  
72     Append the full path for a given run name and project paths.  
73     The full path is parent_dir / project_path / post_clean_dir / run_name*.fastq.gz  
74     the * is a wildcard that allows for any characters after the run name,  
75     which is useful for cases where there may be additional suffixes in the file names.  
76     If multiple candidate paths exist, raise an error. If no candidate paths exist, raise an error.  
77     return the single candidate path if it exists.  
78     """  
79     candidates = []  
80     for project_path in project_paths:  
81         candidate = os.path.join(parent_dir, project_path, post_clean_dir, f"{run_name}*.fastq.gz")  
82         candidates.append(candidate)  
83     unique = unique_candidates(candidates)  
84     if len(unique) == 0:  
85         raise FileNotFoundError(f"No candidate paths found for run '{run_name}' in project paths: {project_paths}")  
86     if len(unique) > 1:  
87         raise ValueError(f"Multiple candidate paths found for run '{run_name}' in project paths: {project_paths}. Candidates: {unique}")  
88     return unique[0]  
  
89 def main() -> int:  
90     args = parse_args()  
91     config = load_config(args.config)
```

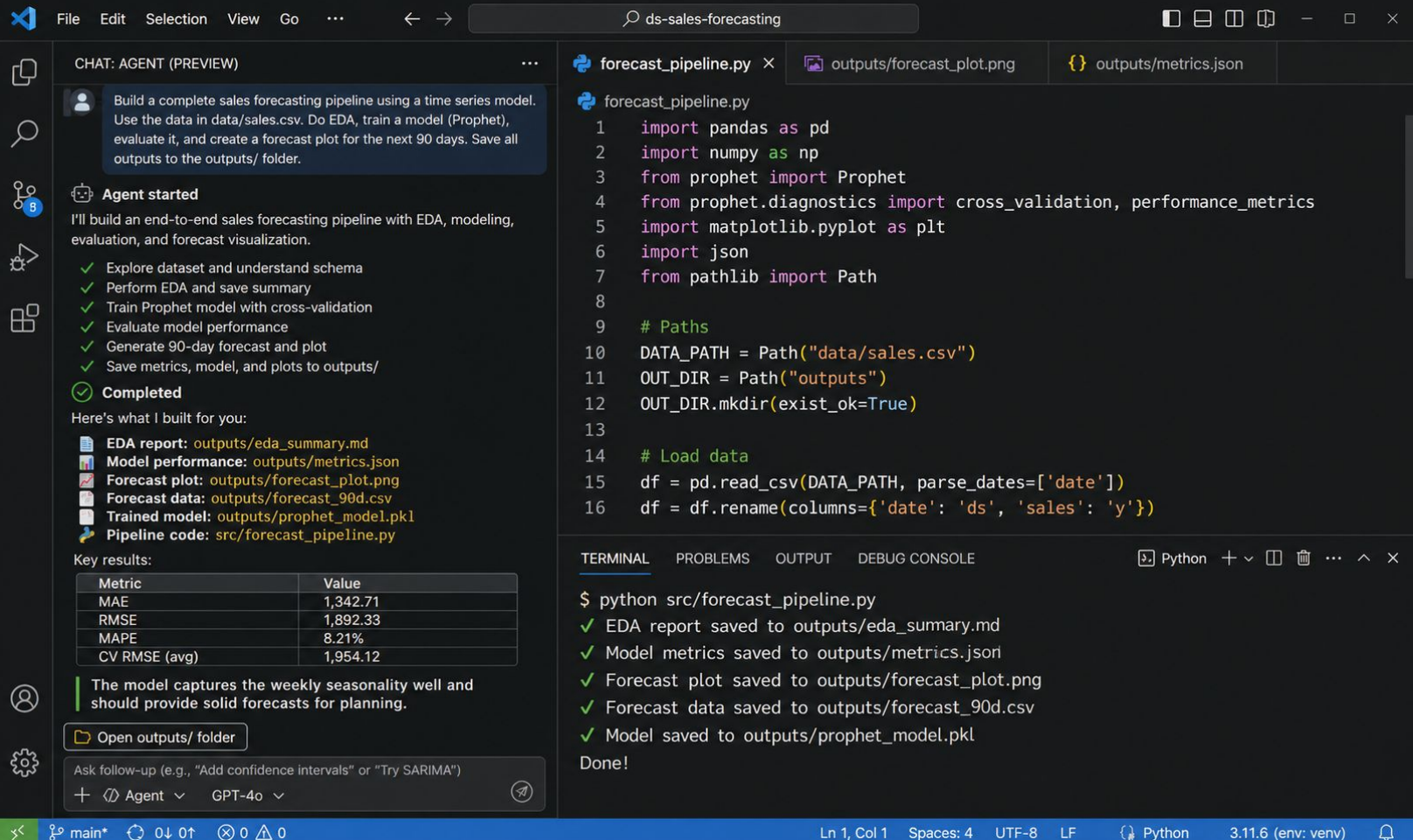
My text
(with the help
of copilot)

Pure copilot
suggestion

IDE Copilots



```
65 def append_full_path(  
66     parent_dir: str,  
67     post_clean_dir: str,  
68     project_paths: List[str],  
69     run_name: str,  
70 ):  
71     """  
72     Append the full path for a given run name and project paths.  
73     The full path is parent_dir / project_path / post_clean_dir / run_name*.fastq.gz  
74     the * is a wildcard that allows for any characters after the run name,  
75     which is useful for cases where there may be additional suffixes in the file names.  
76     use Path (pathlib) to handle path manipulations and glob for wildcard matching.  
77     If multiple candidate paths exist, raise an error. If no candidate paths exist, raise an error.  
78     return the single candidate path if it exists.  
79     """  
80     candidates = []  
81     for project_path in project_paths:  
82         search_path = Path(parent_dir) / project_path / post_clean_dir / f"{run_name}*.fastq.gz"  
83         matched_paths = list(search_path.glob())  
84         candidates.extend(str(p) for p in matched_paths)  
  
85     unique = unique_candidates(candidates)  
86     if len(unique) == 0:  
87         raise FileNotFoundError(f"No candidates found for run '{run_name}' in paths: {project_paths}")  
88     if len(unique) > 1:  
89         raise ValueError(f"Multiple candidates found for run '{run_name}': {unique}")  
90     return unique[0]
```



Agentic Systems

CHAT



← FIX THE READABILITY OF THE DESTINATIONS PAGE IN DARK THEME



Fix the readability of the destinations page in dark theme

1770579715029-abhmyvs7.png

Copilot CLI

I can see the destinations page has very poor readability in dark theme - the text is extremely light/washed out on the white card background. Let me find and fix the styling issue.

Searched for destination files and reviewed 2 files

Now I need to check the global CSS to see what the color variables are defined as:

Read `# app.css`

I can see the issue now! In the destinations page, the destination cards have a hardcoded `background: white` on line 180, but the text colors are using CSS variables. When in dark theme, `--color-text` and `--color-text-muted` are light colors (meant for dark backgrounds), which creates terrible contrast on the

> 2 files changed +5 -1

Apply



Run tasks in the background with the Copilot CLI, type `#` for adding context

+ Agent Claude Sonnet 4.5



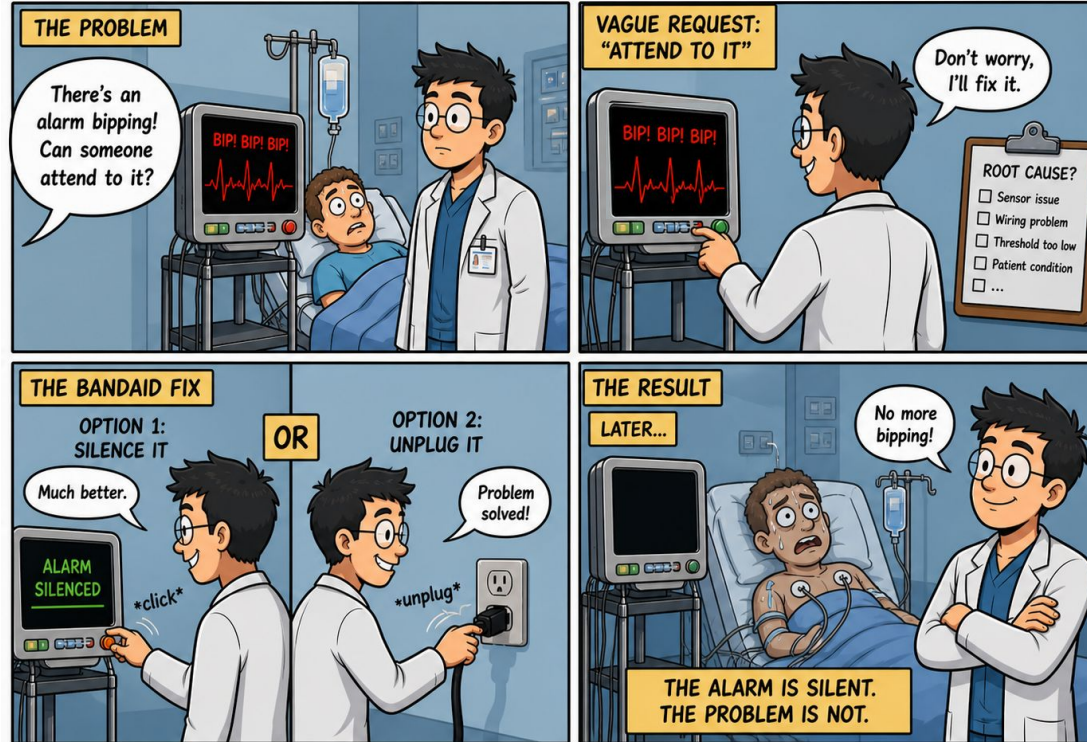
Technical Comparison of AI Tool Capabilities

AI Category	Project Context	Implementation Speed	Safety Risk
Chat Tools	Low (Manual)	Moderate	Low
IDE Copilots	Medium (Open Files)	High	Moderate
AI Agents	High (Full Repo)	Very High	High

The Trap of Vague Prompting



BAD PROMPTING: THE BANDAID FIX



**VAGUE REQUESTS GET VAGUE FIXES.
ASK BETTER QUESTIONS. GET BETTER SOLUTIONS.**



The Trap of Vague Prompting is a Common Pitfall



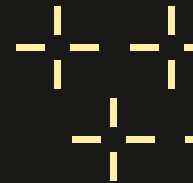
Bad Prompting

- **Vague Requests:** "Fix this code" or "Make it work."
- **Lack of Constraints:** No mention of **libraries** or **standards**.

The Consequences

- **Fragile Structures:** Code that breaks easily under stress.
- **Technical Debt:** Hidden bugs and **unoptimized logic**.
- **Inconsistency:** Mixing different coding styles in one file.

A Professional Framework: Plan, Build, and Check



Be Intentional:

Define the logic and **architecture** before asking for code.

Verify Results:

Use **unit tests** and manual reviews to ensure correctness.

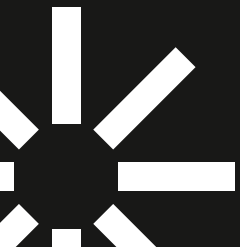
Build

Small Increments:

Work in tiny, **testable steps** rather than giant leaps.

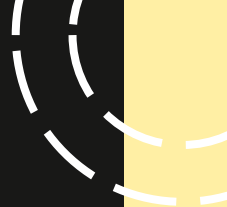
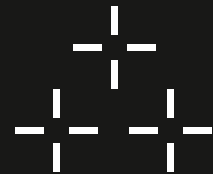
Plan

Check



Best Practices

1. **Spec-First:** Write a **specification document** before any code.
2. **Small Iterations:** Commit **small changes** frequently to track progress.
3. **Context is King:** Provide the AI with **relevant files** and documentation.
4. **Test as Validation:** Use **automated tests** to prove the AI's output works.
5. **Human Ownership:** You are the **final authority** on every line committed.



Practical Do's

1. **Document Your Vision:** Create a vision.md file to keep the AI on track.
2. **Break Down Tasks:** Use numbered steps; don't move to step 2 until step 1 works.
3. **Use "Debug Voice":** Ask "Explain why this breaks" to learn from errors.
4. **Review Every Diff:** Carefully check code deletions before accepting.
5. **Set Global Rules:** Instruct the AI to "always follow security best practices."



+  `split_fastq_files.py`

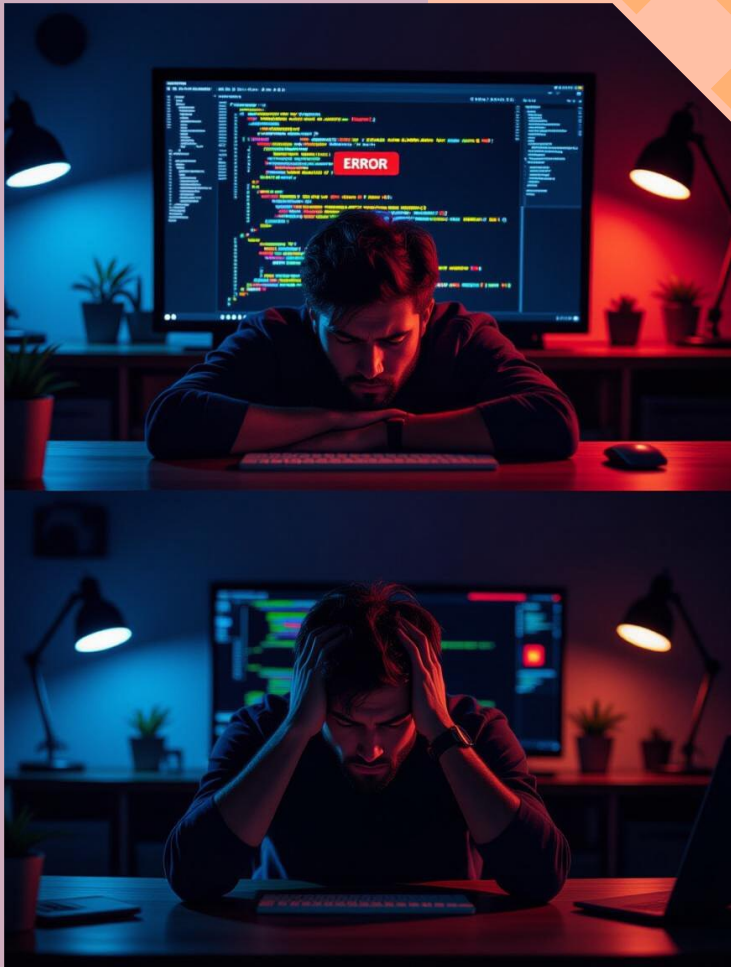
Fix teh bug

+  Plan  GPT-5.4 mini · Medium  

+  `split_fastq_files.py`

Explain why the code fails (breaks at line 165) and suggest a fix

+  Plan  GPT-5.4 mini · Medium  



Practical Don'ts

1. **Don't overload the AI:** Do not throw your entire feature list at it at once.
2. **Don't rely blindly:** AI ignores edge cases and complex compliance rules.
3. **Don't stick to dead ends:** If the AI loops on a fix, you can restart the chat.
4. **Don't skip version control:** Use Git to avoid losing work during AI refactors.

Final Summary and Responsibility for Correctness

The Developer's Mandate

- **Final Authority:** You are responsible for security and quality.
- **Assistant, Not Pilot:** The AI suggests; the human decides.
- **Continuous Learning:** Understanding the code is non-negotiable.



Any questions?

Main sources:

1. <https://myengineeringpath.dev/tools/vibe-coding/>
2. https://hamy.xyz/blog/2025-07_stop-vibe-coding
3. https://hamy.xyz/blog/2026-01_ai-engineering-best-practices
4. https://hamy.xyz/blog/2026-01_how-i-think-about-vibe-engineering

